STUDENT

S-0005-CHZ

TENTAMEN

260107 DT019G DT026G

Kurskod	--
Bedömningsform	--
Starttid	07.01.2026 07:00
Sluttid	07.01.2026 10:00
Bedömningsfrist	--
PDF skapad	15.09.2026 13:02

Instruktioner

Uppgift	Status	Poäng	Uppgiftstyp
ℹ			Information eller resurser

Flervalsuppgifter

Uppgift	Status	Poäng	Uppgiftstyp
1(a)	Delvis rätt	4/5	Flersvarsfråga
1(b)	Rätt	5/5	Flervalsfråga
1(c)	Rätt	5/5	Flervalsfråga
1(d)	Fel	0/5	Flervalsfråga
1(e)	Rätt	5/5	Flersvarsfråga
1(f)	Rätt	5/5	Flervalsfråga
1(g)	Fel	0/5	Flervalsfråga
1(h)	Rätt	5/5	Flervalsfråga
1(i)	Delvis rätt	0/5	Matchning
1(j)	Rätt	5/5	Flervalsfråga
1(k)	Rätt	5/5	Flersvarsfråga
1(l)	Rätt	5/5	Flervalsfråga

Flervalsuppgifter, del 2

Uppgift	Status	Poäng	Uppgiftstyp
ℹ			Information eller resurser
2(a)	Delvis rätt	2/5	Flersvarsfråga

2(b)	Delvis rätt	3/5	Flersvarsfråga

Information inför kompilering

Uppgift	Status	Poäng	Uppgiftstyp
ℹ			Information eller resurser

Programmeringsuppgift 1 (klasser)

Uppgift	Status	Poäng	Uppgiftstyp
3	Rätt	13/15	Kompilering

Programmeringsuppgift 2 (rekursion)

Uppgift	Status	Poäng	Uppgiftstyp
4	Besvarad	0/15	Kompilering

1(a) Vilka av följande rader deklarerar giltiga konstruktorer för klassen Moo?

Välj ett eller flera alternativ:

- [ ] ~Moo();
- [x] Moo(); ✓
- [x] Moo(Moo, Boo) ✓
- [ ] Foo(Moo)
- [x] Moo(Moo) ✗
- [x] Moo(int i = 5) ✓

Totalpoäng: 5

1(b) Varför bör inte `using namespace std;` användas?

Välj ett alternativ:

- Prestandan under *kompilering* blir sämre.
- Prestandan under *exekvering* blir sämre.
- Namnkonflikter kan uppstå med identifierare i andra namnrymder. ✓
- Det kan leda till att särskilda exceptions kastas, även om det är sällsynt.

Totalpoäng: 5

1(c) Varför skapas inte ett nytt objekt i följande kodrad?

```
Maze m;
```

för klassen:

```
class Maze {
public:
 Maze() {}
 Maze(int i = 7, int j = 7) {}
};
```

Välj ett alternativ:

- Konstruktorn är felaktigt inkapslad.
- Anropet till konstruktorn är fel och ska vara: `Maze m();`
- Det finns flera defaultkonstruktorer. ✓
- Det saknas semikolon (`;`) i koden för klassen.
- Det saknas en deklaration för defaultkonstruktorn.

Totalpoäng: 5

1(d) Vilken är den mest sannolika utskriften från en körning av följande kod?

```
std::string s = "Anna";
std::cout << s.substr(0, 1).find("n");
```

Välj ett alternativ:

- 0
- 1 ✗
- 2
- A
- n
- a
- 18446744073709551615 ✓

Totalpoäng: 5

1(e) Vilka av följande påståenden är sanna efter att följande kod exekverat?

```
int *p = new int(-3), *q = new int(3);
p = q;
*q = *p;
delete q;
p = nullptr;
```

Välj ett eller flera alternativ:

- [ ] p blir en dangling pointer.
- [x] q blir en dangling pointer. ✓
- [ ] p blir initialt en wild pointer.
- [ ] q blir initialt en wild pointer.
- [ ] Odefinierat beteende uppstår.
- [x] En minnesläcka uppstår. ✓
- [ ] Bara ett av de tre allokerade talen i `q` blir frigjort.

Totalpoäng: 5

1(f) Vad kännetecknar en koperingskonstruktor?

Välj ett alternativ:

- ( ) Den kan anropas utan argument.
- ( ) Det är den första konstruktorn som definieras för en klass.
- ( ) Den kan endast implementeras implicit (syntetiserad).
- (•) Den tar in en referens till ett objekt av samma klass som parameter. ✓
- ( ) Den skapar tomma objekt.

Totalpoäng: 5

1(g) Varför fungerar inte följande uttryck om `b` är en pekare till ett objekt med medlemsvariabeln `a`?

```
(*b)->a
```

Välj ett alternativ:

- För att `*b` inte är en pekare. ✔
- För att ordningen är omkastad och ska istället vara `(*a)->b`.
- För att `*` syftar till pekare och kan därmed endast användas för pekare.
- För att medlemsoperatorn `->` har högre prioritet än operatorn `*`. ✖
- För att `a` skulle kunna vara en konstant.

Totalpoäng: 5

1(h) Vilken nivå av undantagssäkerhet garanterar att inga resurser läcker och att alla objekt förblir i ett giltigt och helt oförändrat tillstånd efter att ett fel kastats?

Välj ett alternativ:

- No-throw guarantee
- No exception guarantee
- Basic exception guarantee
- Medium exception guarantee
- Strong exception guarantee ✔

Totalpoäng: 5

1(i) Vad kan inte förändras i respektive deklaration av pekaren **`p`**?

```
1 const int* p;
2 int* const p;
3 int const * p;
4 int* p const;
5 const int* const p;
```

Matcha ihop värdena:

	Pekarens lagrade adress får inte ändras	Heltalet som pekas på får inte ändras	Varken pekarens lagrade adress eller heltalet får ändras	Inkorrekt syntax
1	✗	✓		
2	✓			✗
3		✓		✗
4		✗		✓
5			✓	

Totalpoäng: 5

1(j) Anta att du har följande vektor v:

```
std::vector<int> v(8);
```

Anta nu att du lägger till ett element till v. Vilken *kapacitet* (*capacity*) kommer vektorn ha efter tillägget?*

Välj ett alternativ:

- 0
- 1
- 2
- 8
- 9
- 11
- 12 ✓
- 15

** Svaret är implementationsspecifikt och utgår ifrån MSVC men kan även härledas från GCCs och CLANGs implementation.*

Totalpoäng: 5

1(k) Vilka av följande tillhör The Big Three (rule of three)?

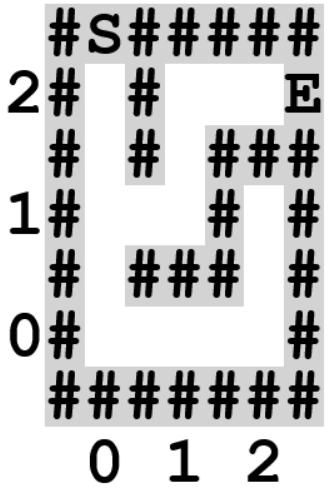
Välj ett eller flera alternativ:

- [ ] Defaultkonstruktorn.
- [ ] Den syntetiserade defaultkonstruktorn.
- [ ] De parametriserade konstruktorerna.
- [x] Kopieringskonstruktorn. ✓
- [x] Destruktorn. ✓
- [ ] In- och utströmsoperatorerna.
- [x] Tilldelningsoperatorn. ✓
- [ ] Subscript-/indexoperatorn.
- [ ] Ingen av de ovanstående.

Totalpoäng: 5

1(I) Följande bild visar en 3x3-labyrint bestående av 9 noder genererad med *depth-first search* (DFS). Vid vilken koordinat startade DFS genereringen av labyrinten?

Koordinaterna är uttrycka på formen (x, y), motsvarande (kolumn, rad).



Välj ett alternativ:

- (0,0)
- (0,1)
- (0,2)
- (1,0)
- (1,1)
- (1,2)
- (2,0)
- (2,1)
- (2,2) ✓

Totalpoäng: 5

2(a) Den rekursiva funktionen **`factorial`** kan användas för att beräkna fakulteten för ett tal, d.v.s. t.ex. $5! = 120$.

```
int factorial(int n) {
    if (n <= 1) { return 1; }

    return n * factorial(n--);
}
```

Vilka problem har koden?

Välj ett eller flera alternativ:

- [ ] Returtypen och inparameterns datatyp bör inte vara **`int`**. ✔
- [ ] Kodraden **`return n * factorial(n-1);`** ska inte ha **`return`**.
- [x] Argumentet är felaktigt i det rekursiva anropet. ✔
- [ ] Basfallet **`n = 0`** hanteras inte.
- [ ] Det finns inget basfall (base case).
- [ ] En stack overflow kommer uppstå vid körning. ✔

Tips:

- Observera att den matematiska funktionen för fakultet endast är definierad för icke-negativa heltal.
- Observera även att $0! = 1$.

Totalpoäng: 5

2(b) Funktionen **`find_in_names`** ska returnera antalet matchningar för needle (namnet som eftersöks) i haystack (vektorn som håller alla personer med namn). Matchningen behöver inte vara exakt utan ska fungera på substrängar så att t.ex. needle "Ann" matchar mot namnen "Anna", "Johanna", "Mannfried", osv. i haystack, vilket då skulle returnera 3 matchningar totalt.

```
int find_in_names(std::vector<Person> haystack, std::string needle) {
    size_t count;
    for (auto person : haystack) {
        if (needle.find(person.name) != std::string::npos) {
            count++;
        }
    }
    return count;
}
```

Vilka problem har funktionen?

Välj ett eller flera alternativ:

- [ ] **`auto person`** skapar en onödig kopia av varje person. En referens bör använda ✔
- [x] **`std::string::find`** är skiftlägeskänslig och kommer inte att matcha t.ex. "Ann" med "Johanna". ✔
- [x] **`count`** är inte initialiserad. ✔
- [x] En konvertering från **`size_t`** till **`int`** sker vilket innebär en möjlig förlust av data. ✔
- [x] Typen är fel i loopen. Den ska vara **`auto Person`** inte **`auto person`**. ✘
- [x] Strängarna är förväxlade i anropet till **`find`**. Funktionen ska anropas från **`person.name`** med **`needle`** som parameter. ✔
- [ ] **`!= std::string::npos`** måste tas bort från villkoret i if-satsen.

Totalpoäng: 5

3 Implementera en klass Book.

Book ska ha följande medlemmar:

- En medlemsvariabel **`title`** av typen std::string.
- En medlemsvariabel **`pages`** av typen unsigned int.
- En medlemsvariabel **`current_page`** av typen unsigned int.
- En parametriserad konstruktor som tar **`title`** och **`pages`** som inparametrar. *Inga andra konstruktorer får förekomma*.
- En funktion **`change_page`** för att bläddra till en given sida.
- En funktion **`next_page`** för att bläddra till nästa sida.
- En funktion **`prev_page`** för att bläddra till föregående sida.
- En överlagrad utströmsoperator (**`<<`**) som skriver ut **`current_page`**.

Den nuvarande sidan får aldrig hamna utanför bokens sidor (**`1` $\leq$ `current_page` $\leq$ `pages`**), vilket måste säkerställas av klassens funktioner som manipulerar **`current_page`**.
Medlemmarna ska inkapslas enligt sedvanlig konvention. Skapa get- och setfunktioner till medlemmarna efter behov.

Input följer formatet:
`title pages start_page`

Output ska ha följande format:
`current_page`
upprepat fyra gånger efter anrop till **`change_page`**, **`next_page`**, **`prev_page`** i sekvens, se kommentarerna nedan.

Testfall #	Input	Förväntad output
1	Hitchhikers 191 100	100 101 100 99
2	TheHobbit 310 310	310 310 309 308
3	HarryPotter 223 1	1 2 1 1

```
#include <iostream>
#include <string>
//GG WP
class Book{
    private:
    std::string title;
    unsigned int pages;
    unsigned int current_page;

    public:
    Book(std::string t, unsigned int p){
        title = t;
        pages = p;
    }
    void change_page(unsigned int pos){
        if (current_page == pages || pos <= pages ){
        current_page = pos;
        }
    }
    void next_page(){
        if(current_page < pages){
        current_page++;
        }
    }
    void prev page(){
```

```
        if(!(current_page <= 1)){
        current_page--;
        }
    }
    friend std::ostream& operator<<(std::ostream& o, const Book& b){
        std::cout << b.current_page;
        return o;
    }
};


int main() {
    std::string a; unsigned int b, c;
    std::cin >> a >> b >> c;

    // skapa en bok genom att anropa konstruktorn med a och b som argument

    Book book(a, b);

    // bläddra till sida c

    book.change_page(c);
    // skriv ut aktuell sida till cout, inklusive radbrytning.
    std::cout << book << std::endl;

    // bläddra till nästa sida
    book.next_page();
    // skriv ut aktuell sida till cout, inklusive radbrytning.
    std::cout << book << std::endl;
    // bläddra till föregående sida
    book.prev_page();
    // skriv ut aktuell sida till cout, inklusive radbrytning.
    std::cout << book << std::endl;

    // bläddra till föregående sida
    book.prev_page();
    // skriv ut aktuell sida till cout, inklusive radbrytning.
    std::cout << book << std::endl;
    return 0;
}
```

Testkod

Observera:

- Globala variabler får inte förekomma.

Totalpoäng: 15

4 Implementera en rekursiv funktion som tar reda på om det finns två tal i en lista som summerar till ett givet mål.

Din indata består av ett antal tal. Det första talet är ditt **mål**. De resterande talen är din **lista** med tal. Du ska alltså ta reda på om det finns två tal i listan som tillsammans adderar till summan som är ditt mål och returnera sant eller falskt därefter.

Exempelvis:
9, 5, 4, 3 → sant, då 5 + 4 = 9.
7, 5, 4, 3 → sant, då 4 + 3 = 7
6, 5, 4, 3 → falskt, då inget tal plus något annat tal i listan har summan 6.
Möjliga summor för [5, 4, 3] är: 5 + 4 = **9**, 5 + 3 = **8**, 4 + 3 = **7**.

Testfall #	Input	Förväntad output
1	9 5 4 3	1
2	7 5 4 3	1
3	6 5 4 3	0

```
// kom ihåg dina includes
#include <iostream>
#include <vector>
bool two_sum(std::vector<int> v, int current_idx, int target) {
    // din lösning här
    //basecase, här tänker jag att summan av index 1 & 2 || 1 & 3 || 2 & 3 = v[0] annars
    if((v[0] != v[1] + v[2]) || (v[0] != v[1] + v[3]) || (v[0] != v[2] + v[3])){
        return false;
    }
    return two_sum()
}

int main() {
    int a,b,c,d;
    std::vector<int> vec = {a,b,c,d};
    while(std::cin>>vec){

    }

    for(int i = 0; i < vec.size(); i++){
        std::cout << vec[i];
    }
    // läs in målet från stdin, därefter övriga tal till din lista, och skriv slutligen
    return 0;
}
//jag är skit på rekursion. Jag förstår frågan, kan bara inte implementera den. Det jag
    OCH anropa samma funktion i funktionen. Just i den här uppgiften så är målet att ind
    1 & 2 || 1 & 3 || 2 & 3, annars når vi inte målet. Om men uppfyller målet så returne
```

Testkod

Observera:

- Funktionen måste returnera en bool.
- Lösningen måste vara rekursiv där rekursionen är av väsentlig betydelse för lösningen.
- Loopar får inte användas för att kringgå rekursionen.

- Globala variabler får inte förekomma.

Tips:

- Du kan loopa igenom input med **`while (std::cin >> temp)`**.
- Det finns många olika tänkbara signaturer. I den föreslagna arbetar du med en vector (listan med talen), ett index för att rekursivt ta dig framåt i listan, och ditt mål (målsumman).
- Du kan ta bort element från början av en vector v med: **`v.erase(v.begin());`**.

Totalpoäng: 15